

İŞLETİM SİSTEMLERİNE GİRİŞ

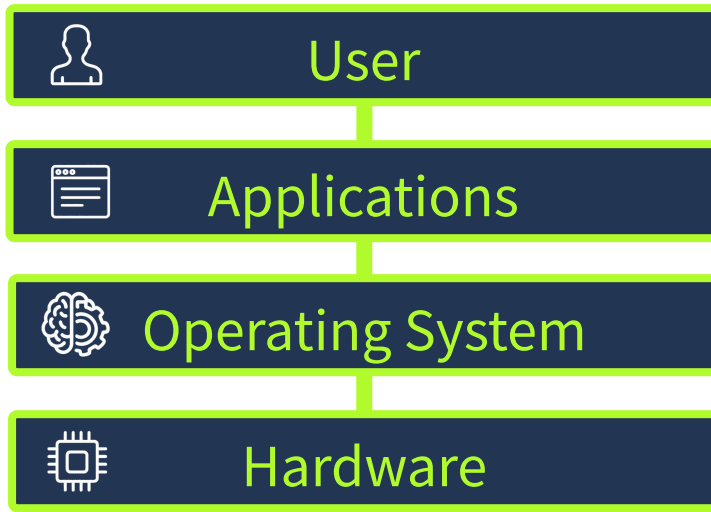
[Kaynak](#)

İÇERİK

1. The Invisible Manager (Görünmez Yönetici)
2. İşletim Sistemi Etkileşimi ve Ekosistemi (OS Interaction and Landscape)

The Invisible Manager (Görünmez Yönetici)

İşletim sistemi (OS), bir bilgisayarda gerçekleşen tüm olayları koordine eden, donanım kaynaklarını yöneten ve kullanıcı ile fiziksel cihaz arasındaki iletişimi sağlayan çekirdek yazılımdır. Modern bilişim dünyasında işletim sistemi, donanım bileşenleri ile son kullanıcı uygulamaları arasında çalışan görünmez bir yönetici, teknik tabirle bir "soyutlama katmanı" (abstraction layer) olarak görev yapar.



Eğer işletim sistemi olmasaydı; her bir yazılımın (web tarayıcısı, oyun vb.) CPU, bellek (RAM), disk ve ağ cihazları üzerinde doğrudan ve bağımsız bir kontrol mekanizması kurması gerekirdi. Bu durum mimari açıdan sürdürülemez bir kaos yaratacak ve donanım çakışmalarına (hardware conflicts) neden olacaktı. İşletim sistemi, kaynak tahsisini (resource scheduling) merkezi olarak yürüterek tüm süreçlerin stabil bir şekilde çalışmasını garanti altına alır.



Sistem Ayrıcalık Katmanları (System Privilege Layers) ve İzolasyon

Modern işletim sistemlerinde mimari, güvenlik ve stabiliteyi sağlamak amacıyla farklı yetki seviyelerine (privilege levels) bölünmüştür. Bir siber güvenlik uzmanı için bu katmanların ayrımı, **Privilege Escalation** (Ayrıcalık Yükseltme - Düşük yetkili bir hesaptan sistem yöneticisi veya kernel yetkilerine geçiş yapma) zafiyetlerini anlamak adına kritik öneme sahiptir. İşletim sistemi temel olarak iki ana hafıza ve yürütme alanına ayrılır:

Kernel Space (Çekirdek Alanı): İşletim sisteminin kalbinin attığı, en yüksek yetkilere sahip ve tamamen izole edilmiş alandır. Donanıma (CPU, RAM, disk) kısıtlamasız ve doğrudan erişim hakkına sahip tek yapıdır. Eğer bir saldırgan bu alanda kod çalıştırabilirse (örneğin bir kernel exploit aracılığıyla), tüm sistemin mutlak kontrolünü ele geçirmiş demektir.

User Space (Kullanıcı Alanı): Standart uygulamaların (tarayıcılar, kelime işlemciler vb.) çalıştığı kısıtlanmış ve güvenli alandır. Bu alanda çalışan hiçbir uygulama doğrudan donanıma müdahale edemez.

ÖNEMLİ: User Space'te çalışan bir uygulamanın donanım kaynağına (örneğin diske dosya yazmak veya ağ üzerinden paket göndermek) ihtiyacı olduğunda, bunu doğrudan yapamaz. Bunun yerine işletim sistemine bir **System Call** (Sistem Çağrısı - Uygulamanın kernel'dan donanım kaynaklarını kullanabilmek için yetki talep ettiği programatik arayüz) yapmak zorundadır. Bu ayrım sayesinde, hatalı kodlanmış veya çöken bir User Space uygulaması tüm işletim sistemini çökertecek (Kernel Panic) bir felakete yol açamaz.

İşletim Sisteminin Çekirdek Sorumlulukları (Core Duties)

İşletim sistemi, arka planda güvenli ve verimli bir çalışma ortamı sağlamak için hayati görevleri üstlenir. Bu görevlerin her biri, sistemin güvenliği ve bütünlüğü için birer yapı taşıdır:

Process Management (Süreç Yönetimi): İşletim sistemi, çalışan programları (süreçleri/process) yaratır, zamanlar (scheduling), önceliklendirir ve sonlandırır. İşlemcinin (CPU) hangi sürece ne kadar zaman ayıracağına karar vererek eşzamanlı çalışmayı (multitasking) mümkün kılar. Güvenlik perspektifinden bakıldığında, zararlı bir yazılımın işlemciyi %100 meşgul ederek sistemi kilitlemesini (Denial of Service - DoS) engellemek bu mekanizmanın görevidir.

Memory Management (Bellek Yönetimi): RAM kaynaklarının süreçler arasında paylaşılmasıdır. İşletim sistemi, her uygulamanın bellekte kendi izole alanında çalışmasını sağlar. Fiziksel RAM dolduğunda, işletim sistemi diskin bir bölümünü **Virtual Memory** (Sanal Bellek) olarak kullanarak sistemin çökmesini engeller. Bellek izolasyonu, bir uygulamanın (örneğin bir oyunun) başka bir uygulamanın (örneğin şifre yöneticisinin) bellek alanını okumasını engelleyen en kritik güvenlik bariyerlerinden biridir.

File System Management (Dosya Sistemi Yönetimi): Verilerin disk üzerinde nasıl organize edileceğini (dizinler, dosya yolları) ve nasıl adlandırılacağını belirler. Dosya boyutu, oluşturulma tarihi gibi **Metadata** (Üst veri - Veri hakkındaki veri) bilgilerini tutar ve dosyalar üzerindeki okuma/yazma/çalıştırma izinlerini yönetir.

User Management (Kullanıcı Yönetimi): Çoklu kullanıcı ortamlarında kimlik doğrulama (Authentication) işlemlerini yönetir. Hangi kullanıcının hangi verilere erişebileceğini belirleyerek sistem içinde yatay (horizontal) yetkisiz erişimleri engeller.

Device Management (Aygıt Yönetimi): Sürücülerini (drivers) yükleyerek yazılımlar ile donanımlar arasında bir **Hardware Abstraction Layer** (Donanım Soyutlama Katmanı - Uygulamaların donanımın marka/modelinden bağımsız standart komutlarla çalışabilmesini sağlayan yapı) oluşturur.

İşletim Sisteminin Yerleşik Güvenlik Temelleri (OS Security Foundation)

Bir sisteme Antivirüs, EDR veya **Firewall** (Güvenlik Duvarı - Ağ trafiğini belirli kurallara göre filtreleyen güvenlik sistemi) kurulmadan önce bile işletim sistemi ilk savunma hattını oluşturur. Güvenlik modeli şu temellere dayanır:

- **Authentication (Kimlik Doğrulama):** Sisteme erişmeye çalışan kişinin, iddia ettiği kişi olup olmadığını parola veya biyometrik verilerle doğrular.
- **Permissions (İzinler):** Kullanıcıların ve uygulamaların hangi dosya ve dizinler üzerinde okuma (Read), yazma (Write) veya çalıştırma (Execute) yetkisi olduğunu katı bir şekilde denetler.
- **Isolation (İzolasyon):** Daha önce bahsedilen Kernel/User Space ayrımı sayesinde her sürecin kendi korumalı alanında (sandboxed) kalmasını sağlar.

- **System Protection (Sistem Koruması):** Kritik sistem dosyalarının ve konfigürasyonlarının (örneğin Windows'ta System32 dizini veya Linux'ta /etc/shadow dosyası) standart kullanıcılar tarafından değiştirilmesini engeller.

Pratik Analiz: Hedef Sistemde İlk Enumeration Aşaması

Siber güvenlikte bir sisteme ilk girildiğinde veya incelenmek üzere ele alındığında yapılan ilk işlem **Enumeration** (Bilgi Toplama - Hedef sistemin mimarisi, çalışan servisleri, donanım kaynakları ve kullanıcıları hakkında detaylı veri çıkarma süreci) adımıdır.

Bize verilen senaryoda, sistemin ne olduğuna dair bilgimiz olmadığı için masaüstündeki **About This Computer** (Bu Bilgisayar Hakkında) kısayolunu kullanarak **System Monitor** (Sistem Monitörü) aracını başlatıyoruz. GUI (Grafiksel Kullanıcı Arayüzü) üzerinden açılan bu sistem monitörü, arka planda sistemin durumunu analiz eden komutların çıktılarını görselleştirir.

Bir sistem monitörü arka planda CPU ve bellek durumunu okumak için teknik olarak işletim sistemine şu tarz komutlar gönderir (Linux bazlı bir sistem olduğu varsayımıyla, arka planda dönen mantık):

Linux

```
# İşletim sistemi arka planda süreçleri ve kaynak tüketimini listeler.  
top -b -n 1 | head -n 15
```

Yukarıdaki komut, System Monitor GUI aracının ekrana yansıttığı verinin terminaldeki karşılığıdır. `top` komutu sistemdeki süreçleri, `-b` parametresi batch modunda (tek seferlik çıktı almak için), `-n 1` parametresi sadece bir iterasyon çalıştırarak çeker. Çıktı `head -n 15` ile borulararak (piping) sadece en çok kaynak tüketen ilk 15 satırın gösterilmesi sağlanır. Bu işlem, Process Management ve Memory Management mekanizmalarının güncel durumunu denetlemek için yapılır.

System Monitor açıldığında odaklanmamız gereken kritik noktalar şunlardır:

1. İşletim sisteminin sürümü ve kernel versiyonu (Hedefte bilinen bir Kernel Zafiyeti olup olmadığını belirlemek için).
2. Arka planda çalışan şüpheli süreçler (Process Management tablosundan anomalileri tespit etmek için).
3. Kaynak tüketimi (Sistemin yük altında olup olmadığını veya bir zararlıının kaynakları sömürüp sömürmediğini anlamak için).

Windows Ortamında Süreç (Process) Analizi ve Kaynak Tüketimi

Linux ortamında `top` aracı ile yaptığımız dinamik süreç izleme işleminin Windows ortamındaki karşılığı, kullanılan komut satırı arayüzüne (CMD veya PowerShell) göre değişiklik gösterir. Windows mimarisi, Linux'un metin tabanlı (text-based) yapısının aksine

nesne tabanlı (object-based) bir yapıya sahip olduğu için, süreçleri ve kaynak tüketimlerini listelerken farklı araçlar ve mantıksal borulama (piping) teknikleri kullanırız.

PowerShell Yaklaşımı (Birebir Karşılık ve Modern Yöntem)

Windows ortamında `top -b -n 1 | head -n 15` komutunun mantıksal olarak en kusursuz karşılığı PowerShell üzerinden gerçekleştirilir. PowerShell, sistemdeki süreçleri birer obje olarak ele aldığı için kaynak tüketimine göre sıralama yapmak çok daha esnek ve analitiktir.

Windows

```
Get-Process | Sort-Object -Property CPU -Descending | Select-Object -First 15
```

Bu komut zincirinin arka plandaki teknik anlamı ve parametre analizi şu şekildedir:

- `Get-Process` : İşletim sisteminde o an aktif olarak çalışan tüm süreçleri (Process) bellek tüketimi, CPU zamanı ve ID (PID) bilgileriyle birlikte nesne olarak çeker.
- `|` (**Pipeline**): Bir önceki komutun ürettiği obje çıktısını, bir sonraki komuta girdi olarak aktarır. Linux'taki kullanım mantığıyla aynıdır.
- `Sort-Object -Property CPU -Descending` : Çekilen süreç listesini, **CPU** (İşlemci tüketimi) özelliğine göre azalan (**-Descending**) bir düzende sıralar. Bu parametre, Linux'taki `top` komutunun varsayılan davranışı olan "en çok kaynak tüketeni en üste al" mantığını simüle eder.
- `Select-Object -First 15` : Tıpkı Linux'taki `head -n 15` gibi, sıralanmış devasa listenin sadece ilk 15 satırını ekrana yazdırır.

Neden bu komutu kullanıyoruz? Bir siber güvenlik uzmanı olarak Windows bir sisteme sızdığımızda (Initial Access) veya bir **Incident Response** (Olay Müdahalesi) vakasında, sistemi yoran şüpheli bir **Cryptominer** (Kripto madenci) zararlı olup olmadığını veya kritik bir sistem sürecine (örneğin `lsass.exe`) **Process Injection** (Süreç Enjeksiyonu - Zararlı kodun yasal bir sürecin bellek alanına aktarılması) yapıp yapılmadığını tespit etmek için anlık kaynak tüketim anomalilerini yakalamak zorundayız.

CMD (Command Prompt) Yaklaşımı ve Kısıtlamalar

Eğer sistemde PowerShell kullanımı kısıtlanmışsa (Örneğin **Constrained Language Mode** aktifse veya Execution Policy engelleri varsa), eski ve geleneksel CMD arayüzüne dönmek bir "B Planı" olarak zorunludur. Ancak CMD'nin yerleşik mimarisi, `top` komutu gibi dinamik bir CPU sıralamasını tek ve basit bir komutla yapmaya uygun değildir. Bunun yerine `tasklist` veya `wmic` kullanılır.

```
tasklist /V | findstr /V "svchost.exe"
```

- **tasklist** : Sistemde çalışan tüm süreçleri, **PID** (Süreç Kimliği), **Session Name** (Oturum Adı) ve **Mem Usage** (Bellek Kullanımı) bilgileriyle listeler.
- **/V (Verbose)**: Bu parametre son derece kritiktir. Çıktıya **Status** (Sürecin durumu) ve en önemlisi **User Name** (Süreci çalıştıran kullanıcı hesabı) bilgilerini ekler. **Privilege Escalation** (Ayrıcalık Yükseltme) aşamasında, NT AUTHORITY\SYSTEM yetkisiyle çalışan zafiyetli bir servis ararken bu parametre hayat kurtarır.
- **findstr /V "svchost.exe"** : Sistemde çok fazla gürültü (noise) yaratan standart Windows servis yöneticisi süreçlerini (**svchost.exe**) ekrandan gizlemek için **-V (Invert Match)** mantığıyla filtreleme yapar.

ÖNEMLİ: **tasklist** komutu varsayılan olarak CPU tüketimine göre sıralama yapamaz. Eğer CMD üzerinden CPU bazlı bir sıralama ve analiz yapılmak zorundaysa, Windows Yönetim Araçları altyapısına (WMI) doğrudan sorgu atan daha kompleks bir komut kullanılmalıdır:

```
wmic path Win32_PerfFormattedData_PerfProc_Process get  
Name,PercentProcessorTime | findstr /V "_Total" | sort /R
```

Bu komut, işletim sisteminin **Kernel Space** (Çekirdek Alanı) seviyesindeki performans sayaçlarına doğrudan erişerek her sürecin yüzde kaç işlemci harcadığını çeker ve **sort /R** (Reverse - Tersine sırala) ile en çok kaynak tüketeni en üste yerleştirir. İşletim sisteminin derinliklerindeki performans objelerini (Win32_PerfFormattedData) doğrudan sorguladığı için **top** komutunun teknik olarak en ham (raw) CMD karşılığı budur.

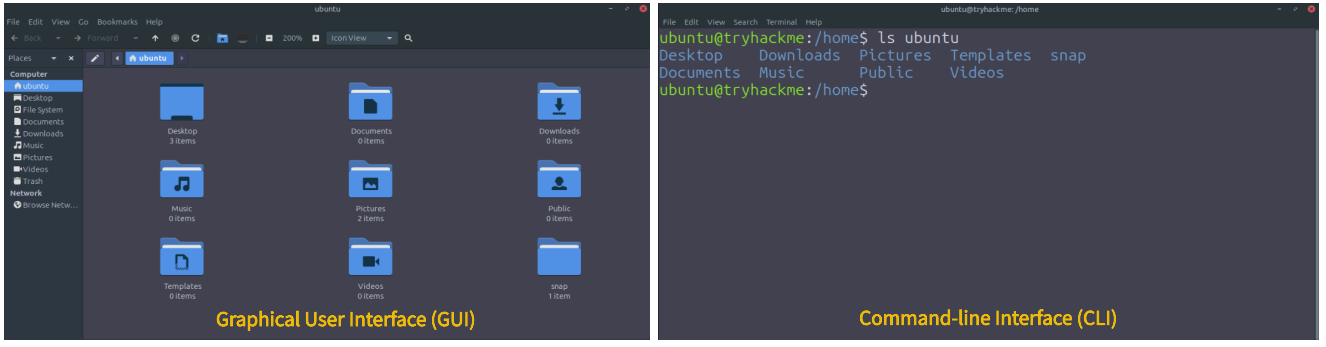
İşletim Sistemi Etkileşimi ve Ekosistemi (OS Interaction and Landscape)

İşletim sisteminin arka plandaki yöneticilik görevlerini anladıktan sonra, bir siber güvenlik uzmanının bu çekirdek yapıyla nasıl iletişim kurduğunu kavramak hayati önem taşır. Hedef sistemin mimarisine, kullanım amacına ve bize sunduğu erişim seviyesine göre işletim sistemiyle iki ana yöntem üzerinden etkileşime geçeriz: **GUI** ve **CLI**.

İşletim Sistemi Arayüzleri (OS Interfaces)

Graphical User Interface (GUI - Grafiksel Kullanıcı Arayüzü): İkonlar, pencereler ve menüler üzerinden fare tıklamalarıyla sistemin yönetildiği, son kullanıcı odaklı arayüzdür. Arka planda çalışan karmaşık komutları görsel bir soyutlama ile gizler.

Command-line Interface (CLI - Komut Satırı Arayüzü): Kullanıcının işletim sistemine doğrudan, metin tabanlı komutlar ve spesifik argümanlar (parametreler) gönderdiği yapıdır. Siber güvenlikte mutlak hakimiyet alanı burasıdır.



ÖNEMLİ: Sızma testlerinde (Penetration Testing) veya gerçek dünya saldırılarında bir hedefte kod çalıştırdığınızda (RCE - Remote Code Execution), sistem size asla bir GUI (masaüstü ortamı) sunmaz. Çoğu zaman bir **Reverse Shell** (Ters Kabuk - Hedef makinenin saldırganın makinesine komut satırı bağlantısı başlattığı oturum) veya **Bind Shell** üzerinden sadece ham bir CLI elde edersiniz. Bu nedenle sistemi klasör tıklayarak değil, komutların sentaksını ve bayraklarını (flags) ezbere bilerek yönetmek zorundasınız.

Bize verilen eğitim metninde, `ubuntu` kullanıcısının **Home** (Ev) dizinindeki dosyaları listelemek için GUI ve CLI kıyaslaması yapılmıştır. Bu işlemin GUI'de birkaç fare tıklaması gerektirmesine karşın, CLI üzerindeki tam ve teknik karşılığı aşağıdaki gibidir:

```
# Sadece dosyaları listelemekle kalmayıp, gizli dosyaları ve detaylı izinleri (permissions) görmek için kullanılır.  
ls -la /home/ubuntu/
```

Bu komuttaki `ls` (list) aracı dizin içeriğini okur. `-l` (long format) parametresi dosya sahipliğini, boyutunu ve okuma/yazma/çalıştırma izinlerini detaylandırır. Kırmızı takım (Red Team) için kritik olan `-a` (all) parametresi ise, standart GUI'de gizlenen ve genellikle SSH anahtarlarının veya bash geçmişinin (history) bulunduğu nokta (`.`) ile başlayan dosyaları (Örn: `.ssh` , `.bashrc`) görünür kılar.

İşletim Sistemi Kategorileri (The OS Landscape)

Sektörde karşılaştığımız her cihaz aynı işletim sistemi mimarisiyle çalışmaz. Hedefin rolü (bir veri tabanı sunucusu mu yoksa bir akıllı telefon mu olduğu), sömürü (exploitation) vektörlerimizi tamamen değiştirir. Mimari açıdan işletim sistemleri beş ana kategoriye ayrılır:

1. Desktop (Masaüstü İşletim Sistemleri): Kişisel bilgisayarlar için tasarlanmıştır. Odak noktası zengin bir GUI, aynı anda birden çok uygulamanın sorunsuz çalışması ve son kullanıcı deneyimidir.

- **Ailesi:** Windows (10, 11), macOS (Sonoma, Sequoia vb.) ve son kullanıcı odaklı Linux dağıtımları (Distributions) olan Ubuntu, Fedora, Debian.
- **Güvenlik Perspektifi:** Bu sistemlerde genellikle zafiyetler, **Phishing** (Oltalama) yoluyla kullanıcının zararlı bir dosyayı çalıştırması veya yerel ağdaki (LAN) zayıf protokoller üzerinden sömürülür.

2. Server (Sunucu İşletim Sistemleri): Veri merkezlerinde, bulut altyapılarında veya şirket ağlarında barındırma, veri tabanı yönetimi ve ağ servisleri sağlamak için tasarlanmıştır. En büyük özellikleri **Headless** (Başsız - GUI barındırmayan, sadece uzaktan erişimle ve komut satırıyla yönetilen) mimaride olmaları ve kesintisiz çalışma (uptime) süreleridir.

- **Ailesi:** Windows Server (2016, 2019, 2022, 2025), Linux (Ubuntu Server, CentOS, Red Hat Enterprise Linux) ve devasa kurumsal altyapılarda kullanılan Unix (IBM AIX, Oracle Solaris).
- **Güvenlik Perspektifi:** Dış dünyaya (İnternete) açık oldukları için genellikle açık portlar, zafiyetli web uygulamaları veya yanlış yapılandırılmış servisler (Misconfigurations) üzerinden saldırıya uğrarlar.

3. Mobile (Mobil İşletim Sistemleri): Akıllı telefonlar ve tabletler için güç verimliliği ve dokunmatik arayüz odaklı tasarlanmıştır.

- **Ailesi:** Android (14-16) ve iOS.
- **Güvenlik Perspektifi:** Bu sistemlerin çekirdek güvenlik felsefesi **App Sandboxing** (Uygulama İzolasyonu - Her uygulamanın işletim sisteminin geri kalanından ve diğer uygulamalardan izole edilmiş kısıtlı bir ortamda çalışması) mimarisine dayanır. Bir zararlının sistemi tam anlamıyla ele geçirebilmesi için bu kum havuzundan kaçması (Sandbox Escape) ve Root/Jailbreak yetkilerine ulaşması gerekir.

4. Embedded ve IoT (Gömülü Sistemler ve Nesnelerin İnterneti): Yönlendiriciler (Routers), akıllı ev aletleri veya endüstriyel makineler gibi sınırlı donanımla tek bir spesifik görevi yerine getirmek üzere sıkıştırılmış (Tiny footprint) sistemlerdir.

- **Ailesi:** OpenWrt, Ubuntu Core. Özellikle havacılık veya endüstriyel kontrol sistemleri (SCADA) gibi milisaniyelik tepki sürelerinin hayati olduğu yerlerde **Real-Time OS (RTOS - Gerçek Zamanlı İşletim Sistemi)** olan FreeRTOS veya VxWorks kullanılır.
- **Güvenlik Perspektifi:** Bu cihazlar genellikle yıllarca güncellenmezler. Bu nedenle varsayılan parolalar (Default credentials) ve eski, bilinen zafiyetler barındıran servis komutları içerirler.

5. Virtual ve Cloud (Sanal ve Bulut Sistemleri): Devasa veri merkezlerinde kaynakları paylaşmak ve uygulamaları hızla yayına almak (Rapid deployment) için tasarlanmış, ultra hafif mimarilerdir.

- **Ailesi:** Cloud/VM altyapılarında Ubuntu LTS, Amazon Linux kullanılırken; **Container-optimized** (Konteyner optimize - Sadece uygulamanın çalışması için gereken minimal kütüphaneleri barındıran yapılar) ortamlarda Alpine Linux, Flatcar Linux gibi sistemler tercih edilir.
- **Güvenlik Perspektifi:** Alpine Linux gibi sistemler o kadar küçültülmüştür ki, sızdığınızda standart bir `bash` kabuğu veya dosya indirmek için kullanacağınız `curl` komutu bile sistemde yüklü olmayabilir. Bu ortamlarda **Living off the Land (LotL - Sistemdeki**

mevcut ve yasal araçları saldırı amacıyla kullanma) teknikleri sınırlanır, B planı olarak statik derlenmiş binary dosyaları hedef sisteme aktarmanız gerekir.

İleri Düzey Araştırma: Hedef Sistemde Derinlemesine Enumeration

Eğitim platformundaki senaryoda, sistemin **Ubuntu Linux** olduğunu öğrendik.

Masaüstündeki "About This Computer" arayüzünü kullanmak yerine, gerçek bir senaryoda uzaktan CLI erişimi elde ettiğimizi varsayarak bu bilgileri nasıl doğrulayacağımızı ve **Home** dizininde nasıl analiz yapacağımızı simüle edelim.

Hedefin dağıtımını (Distribution) ve sürüm numaralarını (Release) işletim sisteminin kalbinden, konfigürasyon dosyalarından okuyarak kesinleştiririz:

```
# İşletim sisteminin sürüm ve mimari bilgilerini barındıran sistem dosyasını  
ekrana yazdırır.  
cat /etc/os-release
```

*cat (concatenate) komutu, metin tabanlı dosyaların içeriğini terminal ekranına doğrudan basar. /etc/os-release dosyası, tüm modern Linux dağıtımlarında işletim sisteminin adını, versiyon numarasını ve kod adını standartlaştırılmış bir formatta tutar. Bu komutun çıktısında "VERSION=22.04 LTS" gibi bir ibare gördüğümüzde, hedefin hangi kernel açıklarına veya **Privilege Escalation** zafiyetlerine sahip olabileceğini tespit etmek için Exploit veri tabanlarında doğru aramayı yapabiliriz.*

Ardından hedefin kullanıcı bağlamını (User Context) ve kişisel çalışma alanını (Home directory) incelemek için aşağıdaki dizin değiştirme ve okuma işlemini yaparız:

```
# Kullanıcının başlangıç (Home) dizinine geçiş yapar ve gizli/normal tüm  
içeriği detaylı listeler.  
cd ~ && ls -la
```

cd ~ (change directory), hedeflenen path'i yazmaya gerek kalmadan o anki aktif kullanıcının home dizinine (örneğin /home/ubuntu) doğrudan atlamayı sağlar. && (AND operatörü), ilk komut (cd ~) hata vermeden başarılı olursa, hemen ardından ikinci komutun (ls -la) çalıştırılmasını sağlayan mantıksal bir zincirleyicidir. Eğer cd komutu yetki eksikliğinden başarısız olursa, yanlış dizinde ls yapmamızı engelleyerek temiz bir çalışma alanı sunar.